

# MapDrawTool — Bug Fix (Square Shape Crash)

```

/**
 * @file MapDrawTool.jsx
 * @description Interactive map drawing tool for FirstKnock.
 *
 * Supports three draw modes:
 *   - 'circle' : click center, click edge !' circular GeoJSON polygon
 *   - 'polygon' : click multiple points, double-click to close !' freehand polygon
 *   - 'square' : click center, click edge !' Mercator-corrected square polygon
 *
 * BUG FIXES applied vs. original MapDrawTool:
 *   1. Added explicit 'square' case in draw handler (was missing !' fell through to
 *      undefined !' TypeError: handler is not a function)
 *   2. Square polygon uses Mercator-corrected lngDelta so it looks square at all
 *      latitudes (was using raw lat delta for both axes !' distorted rectangle)
 *   3. Square polygon is properly closed (5 points: 4 corners + repeat of corner[0])
 *      (was 4 points !' turf.booleanPointInPolygon threw on unclosed ring)
 *   4. Minimum size guard: squares < ~50 m per side are silently ignored
 *      (was missing !' accidental taps produced degenerate 0-area polygons)
 *   5. GeoJSON Feature output format is consistent across all three modes
 */

import React, { useState, useEffect, useRef, useCallback } from 'react';

// -----
// Constants
// -----

/** Minimum square side length in miles (~50 metres). */
const MIN_SQUARE_SIDE_MILES = 0.031;

/** Number of points used to approximate a circle polygon. */
const CIRCLE_POINTS = 64;

// -----
// Helpers
// -----

/**
 * Haversine distance in miles between two lat/lng points.
 * @param {number} lat1
 * @param {number} lng1
 * @param {number} lat2
 * @param {number} lng2
 * @returns {number} Distance in miles
 */
function haversineDistanceMiles(lat1, lng1, lat2, lng2) {
  const R = 3959;

```

```

const dLat = (lat2 - lat1) * (Math.PI / 180);
const dLng = (lng2 - lng1) * (Math.PI / 180);
const a =
  Math.sin(dLat / 2) ** 2 +
  Math.cos(lat1 * (Math.PI / 180)) *
    Math.cos(lat2 * (Math.PI / 180)) *
    Math.sin(dLng / 2) ** 2;
return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
}

/**
 * Build a closed GeoJSON Polygon ring for a circle approximated by N points.
 * @param {number} centerLat
 * @param {number} centerLng
 * @param {number} radiusMiles
 * @param {number} [numPoints=64]
 * @returns {number[][]} Array of [lng, lat] coordinate pairs (closed ring)
 */
function buildCircleRing(centerLat, centerLng, radiusMiles, numPoints = CIRCLE_POINTS) {
  const coords = [];
  for (let i = 0; i <= numPoints; i++) {
    const angle = (i / numPoints) * 2 * Math.PI;
    // BUG FIX (2): Use Mercator-corrected lngDelta for circle too, for consistency
    const latDelta = radiusMiles / 69;
    const lngDelta = radiusMiles / (69 * Math.cos(centerLat * (Math.PI / 180)));
    const lat = centerLat + latDelta * Math.sin(angle);
    const lng = centerLng + lngDelta * Math.cos(angle);
    coords.push([lng, lat]);
  }
  return coords;
}

/**
 * Build a closed GeoJSON Polygon ring for a Mercator-corrected square.
 *
 * BUG FIX (2): Raw lat/lng deltas produce a distorted rectangle at non-equatorial
 * latitudes because 1° of longitude is shorter than 1° of latitude. Correct by
 * dividing the longitude delta by cos(lat) so the square looks visually square.
 *
 * BUG FIX (3): Ring must be closed — the 5th point repeats the 1st corner.
 * Without this, turf.booleanPointInPolygon throws "ring is not closed".
 *
 * @param {number} centerLat Center latitude
 * @param {number} centerLng Center longitude
 * @param {number} halfSideMiles Half the side length in miles
 * @returns {number[][]} Array of 5 [lng, lat] coordinate pairs (closed ring)
 */

```

[illegible]

```
const dblClickListenerRef = useRef(null);
```

```
// %% Cleanup helpers %%%%%%%%%%
```

```
const removeListeners = useCallback(() => {
  if (clickListenerRef.current) {
    window.google?.maps?.event?.removeListener(clickListenerRef.current);
    clickListenerRef.current = null;
  }
  if (dblClickListenerRef.current) {
    window.google?.maps?.event?.removeListener(dblClickListenerRef.current);
    dblClickListenerRef.current = null;
  }
}, []);
```

```
const removePreview = useCallback(() => {
  if (previewOverlayRef.current) {
    previewOverlayRef.current.setMap(null);
    previewOverlayRef.current = null;
  }
}, []);
```

```
const resetState = useCallback(() => {
  setFirstClick(null);
  setPolygonPoints([]);
  removePreview();
}, [removePreview]);
```

```
// %% Reset when mode changes or tool deactivated %%%%%%%%%%
```

```
useEffect() => {
  resetState();
  removeListeners();
}, [drawMode, isActive, resetState, removeListeners]);
```

```
// %% Draw a preview polygon on the map %%%%%%%%%%
```

```
const showPreview = useCallback(
  (coordinates) => {
    if (!map || !window.google) return;
    removePreview();

    const path = coordinates.map(([lng, lat]) => ({ lat, lng }));
    previewOverlayRef.current = new window.google.maps.Polygon({
      paths: path,
      strokeColor: '#FF6B35',
      strokeOpacity: 0.9,
      strokeWeight: 2,
```

```
fillColor: '#FF6B35',
fillOpacity: 0.15,
map,
});
},
[map, removePreview],
);
```

```
// %% Circle handler %%%%%%%%%%
```

```

/**
 * Handle a map click in circle mode.
 * First click = center; second click = edge (defines radius).
 * @param {google.maps.MapMouseEvent} event
 */
const handleCircleClick = useCallback(
  (event) => {
    const lat = event.latLng.lat();
    const lng = event.latLng.lng();

    if (!firstClick) {
      setFirstClick({ lat, lng });
      return;
    }

    const radiusMiles = haversineDistanceMiles(firstClick.lat, firstClick.lng, lat, lng);
    if (radiusMiles < MIN_SQUARE_SIDE_MILES / 2) {
      // Accidental tap — ignore
      resetState();
      return;
    }

    const ring = buildCircleRing(firstClick.lat, firstClick.lng, radiusMiles);
    showPreview(ring);

    /** @type {GeoJSON.Feature} */
    const feature = {
      type: 'Feature',
      geometry: {
        type: 'Polygon',
        coordinates: [ring],
      },
      properties: {
        shapeType: 'circle',
        radiusMiles,
      },
    };
  },
);

```

```
onDrawComplete(feature);
resetState();
},
[firstClick, onDrawComplete, resetState, showPreview],
);
```

```
// %% Polygon handler %%%%%%%%%%
```

```
/**
 * Handle a map click in polygon (freehand) mode.
 * Single clicks add vertices; double-click closes the polygon.
 * @param {google.maps.MapMouseEvent} event
 */
```

```
const handlePolygonClick = useCallback(
  (event) => {
    const lat = event.latLng.lat();
    const lng = event.latLng.lng();
    setPolygonPoints((prev) => [...prev, [lng, lat]]);
  },
  [],
);
```

```
/**
 * Handle a map double-click in polygon mode — closes and emits the polygon.
 * @param {google.maps.MapMouseEvent} event
 */
```

```
const handlePolygonDbClick = useCallback(
  (event) => {
    setPolygonPoints((prev) => {
      if (prev.length < 3) {
        resetState();
        return [];
      }
      // Close the ring
      const ring = [...prev, prev[0]];
      showPreview(ring);
    });
  },
  [resetState, showPreview]
);
```

```
/** @type {GeoJSON.Feature} */
const feature = {
  type: 'Feature',
  geometry: {
    type: 'Polygon',
    coordinates: [ring],
  },
  properties: {
    shapeType: 'polygon',
  }
}
```

[illegible]





```

clickHandler = handleCircleClick;
} else if (drawMode === 'polygon') {
  clickHandler = handlePolygonClick;
} else if (drawMode === 'square') {
  // BUG FIX (1): Explicit square handler — was missing entirely
  clickHandler = handleSquareClick;
} else {
  // Unknown mode — do nothing
  return;
}

clickListenerRef.current = map.addListener('click', clickHandler);

if (drawMode === 'polygon') {
  dblClickListenerRef.current = map.addListener('dblclick', handlePolygonDbClick);
}

return () => {
  removeListeners();
};
}, [
  map,
  isActive,
  drawMode,
  handleCircleClick,
  handlePolygonClick,
  handleSquareClick,
  handlePolygonDbClick,
  removeListeners,
]);

```

```
// %% Update polygon preview as user clicks %%%%%%%%%%
```

```
useEffect(() => {
  if (drawMode !== 'polygon' || polygonPoints.length < 2) return;
  // Show open polyline preview (not closed yet)
  showPreview([...polygonPoints, polygonPoints[0]]);
}, [drawMode, polygonPoints, showPreview]);
```

```
// %% Render %%%%%%%%%%%%%%
```

```
if (!isActive) return null;
```

```
return (
  <div
    style={{
      position: 'absolute',
```

```

top: 12,
left: '50%',
transform: 'translateX(-50%)',
background: 'rgba(0,0,0,0.72)',
color: '#fff',
borderRadius: 8,
padding: '6px 16px',
fontSize: 13,
fontWeight: 600,
pointerEvents: 'none',
zIndex: 10,
letterSpacing: 0.3,
}}
>
{drawMode === 'circle' && (
  firstClick
    ? '⌀=ÜÍ Tap to set circle edge'
    : '⌀=ÜÍ Tap to set circle center'
)}
{drawMode === 'square' && (
  firstClick
    ? '⌀=ÜÐ Tap to set square size'
    : '⌀=ÜÐ Tap to set square center'
)}
{drawMode === 'polygon' && (
  polygonPoints.length === 0
    ? "p Tap to start polygon"
    : `p ${polygonPoints.length} point${polygonPoints.length > 1 ? 's' : ''} — d
)}
</div>
);
}

```